25 Kesalahan Umum dalam Membuat Class Diagram

©PRODI. D4 T. TELEKOMUNIKASI, 2025

Klasifikasi Kesalahan dalam Membuat Activity Diagram

- Kesalahan dalam Prinsip Desain Objek
- Kesalahan dalam Penggunaan Relasi Antar Kelas
- Kesalahan dalam Struktur Diagram dan Representasi Informasi

Klasifikasi Kesalahan dalam Membuat Activity Diagram (Cont.)

- Kesalahan dalam Pemilihan dan Pengorganisasian *Class*
- Kesalahan dalam Penggunaan Interface dan Pemisahan Tugas
- Kesalahan dalam Penamaan dan Konsistensi

Kesalahan dalam Prinsip Desain Objek

- **Mengabaikan Enkapsulasi**

  Tidak membedakan visibility (public, private, protected) pada atribut dan metode, menyebabkan desain tidak aman. [1]

- **Terlalu Banyak Tanggung Jawab**

  Satu kelas melakukan terlalu banyak tugas, melanggar prinsip Single Responsibility. [2]

- **Tidak Menggunakan Generalisasi**

  Mengulang atribut/metode yang seharusnya diwariskan dari superclass. [4]

Kesalahan dalam Prinsip Desain Objek (cont.)

- **Duplikasi Atribut**

  Atribut yang sama dideklarasikan di banyak kelas yang seharusnya diwarisi. [9]

- **Menjadikan Semua Atribut Public**

  Mengabaikan prinsip encapsulation dengan membuat semua atribut dapat diakses langsung. [10]

- **Terlalu Banyak Pewarisan**

  Hierarki terlalu dalam, menyulitkan pemahaman dan perawatan. [18]

- **Tidak Memperhatikan Pola Desain**

  Class diagram tidak memanfaatkan design pattern yang relevan. [22]

Kesalahan dalam Penggunaan Relasi Antar Kelas

- **Penggunaan Asosiasi yang Tidak Perlu**

  Menghubungkan semua kelas dengan asosiasi meskipun tidak saling bergantung. [3]

- **Tidak Menyediakan Relasi yang Jelas**

  Relasi antar kelas tidak diberi arah atau label yang jelas. [6]

- **Mengabaikan Multiplicity**

  Tidak mendefinisikan jumlah objek yang terlibat dalam relasi (1..*, 0..1, dll). [7]

- **Hubungan yang Tidak Simetris**

  Satu sisi relasi memiliki nama/arah, sisi lainnya tidak dijelaskan. [13]

Kesalahan dalam Penggunaan Relasi Antar Kelas (cont.)

- **Menggunakan Agregasi di Tempat yang Tidak Perlu**

  Agregasi digunakan meskipun hubungan hanya asosiasi biasa. [14]
- **Tidak Menamai Relasi**

  Relasi antar class tidak dinamai, membuat maknanya tidak jelas. [15]
- **Tidak Menggambarkan Komposisi Saat Diperlukan**

  Hubungan kepemilikan penuh tidak digambarkan dengan benar. [21]
- **Tidak Mengidentifikasi Dependency**

  Ketergantungan antar class tidak digambarkan. [23]

Kesalahan dalam Struktur Diagram dan Representasi Informasi

- **Menambahkan Logika Bisnis dalam Class Diagram**

  Class diagram hanya menggambarkan struktur, bukan perilaku detail. [5]
- **Terlalu Banyak Atribut dalam Satu Class**

  Class terlalu besar dan sulit dipelihara. [16]
- **Tidak Menentukan Tipe Data Atribut**

  Atribut tidak memiliki tipe data, membuat diagram tidak informatif. [17]
- **Terlalu Abstrak atau Terlalu Detail**

  Diagram tidak proporsional dalam kompleksitas dan cakupan. [24]

Kesalahan dalam Struktur Diagram dan Representasi Informasi (cont.)

- **Tidak Menambahkan Keterangan Tambahan**
  - Diagram tanpa catatan atau dokumentasi membuatnya sulit dimengerti. [25]
  - Diagram yang hanya menunjukkan nama kelas dan atribut tanpa penjelasan tentang tipe data, relasi, atau visibilitas akan menyulitkan pengembang saat mentranslasikan ke dalam kode nyata. [26]
  - Tanpa keterangan, satu diagram bisa ditafsirkan berbeda oleh masing-masing pengembang, yang menyebabkan ketidakkonsistenan dalam pengembangan sistem. [27]

Kesalahan dalam Pemilihan dan Pengorganisasian Class

- **Membuat Class yang Tidak Relevan**

  Terlalu banyak class yang tidak memiliki peran signifikan dalam sistem. [11]
- **Tidak Konsisten dengan Domain**

  Class tidak merepresentasikan konsep nyata dari domain masalah. [12]
- **Tidak Memisahkan Class Entitas, Boundary, dan Control**

  Class diagram bercampur antara logic, interface, dan entitas. [19]

Kesalahan dalam Penggunaan Interface dan Pemisahan Tugas

- **Tidak Menggunakan Interface**
  - Semua fungsi didefinisikan dalam class konkret, tanpa pemisahan antarmuka. [8]
  - Jika suatu class diagram tidak menggunakan interface, maka pengganti atau perluasan dari suatu class akan membutuhkan perubahan struktur yang signifikan pada keseluruhan sistem, sehingga akan mengurangi fleksibelitas sistem. [29]

Kesalahan dalam Penamaan dan Konsistensi

- **Penggunaan Nama yang Tidak Konsisten**

  Penamaan class dan atribut tidak konsisten atau tidak jelas. [20]

Referensi daftar Pustaka

1. Larman, C. (2005). Applying UML and Patterns. Prentice Hall.
2. Martin, R. C. (2002). Agile Software Development: Principles, Patterns, and Practices.
3. Fowler, M. (2003). UML Distilled: A Brief Guide to the Standard Object Modeling Language.
4. Booch, G., Rumbaugh, J., & Jacobson, I. (2005). The Unified Modeling Language User Guide.
5. Ambler, S. W. (2004). The Elements of UML 2.0 Style.
6. Arlow, J., & Neustadt, I. (2005). UML 2 and the Unified Process.

Referensi daftar Pustaka (cont.)

7. Bennett, S., McRobb, S., & Farmer, R. (2006). Object-Oriented Systems Analysis and Design.
8. Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). Design Patterns.
9. Whitten, J. L., & Bentley, L. D. (2007). Systems Analysis and Design Methods.
10. Sommerville, I. (2011). Software Engineering.
11. Pressman, R. S. (2014). Software Engineering: A Practitioner's Approach.
12. Jacobson, I., Christerson, M., Jonsson, P., & Overgaard, G. (1992). Object-Oriented Software Engineering.

Referensi daftar Pustaka (cont.)

13. Blaha, M., & Rumbaugh, J. (2005). Object-Oriented Modeling and Design with UML.
14. Pilone, D., & Pitman, N. (2005). UML 2.0 in a Nutshell.
15. Schneider, D., & Winters, G. (2001). Applying Use Cases: A Practical Guide.
16. Dennis, A., Wixom, B. H., & Roth, R. M. (2015). Systems Analysis and Design.
17. Pooley, R., & Stevens, P. (1999). Using UML: Software Engineering with Objects and Components.
18. Riel, A. J. (1996). Object-Oriented Design Heuristics.

Referensi daftar Pustaka (cont.)

19. Shlaer, S., & Mellor, S. J. (1988). Object-Oriented Systems Analysis.
20. Yourdon, E. (1994). Modern Structured Analysis.
21. Douglass, B. P. (1999). Doing Hard Time: Developing Real-Time Systems with UML.
22. Buschmann, F., Meunier, R., Rohnert, H., Sommerlad, P., & Stal, M. (1996). Pattern-Oriented Software Architecture.
23. Ghezzi, C., Jazayeri, M., & Mandrioli, D. (2002). Fundamentals of Software Engineering.

Referensi daftar Pustaka (cont.)

24. Balagurusamy, E. (2008). Object Oriented Programming with C++.
25. Wiegers, K. E., & Beatty, J. (2013). Software Requirements.
26. Fowler, 2004, UML Distilled: A Brief Guide to the Standard Object Modeling Language
27. Booch, Rumbaugh & Jacobson, 2005, The Unified Modeling Language User Guide
28. Martin, R. C., 2002, Agile Software Development, Principles, Patterns, and Practices
29. Gamma et al., 1994, Design Patterns: Elements of Reusable Object-Oriented Software

